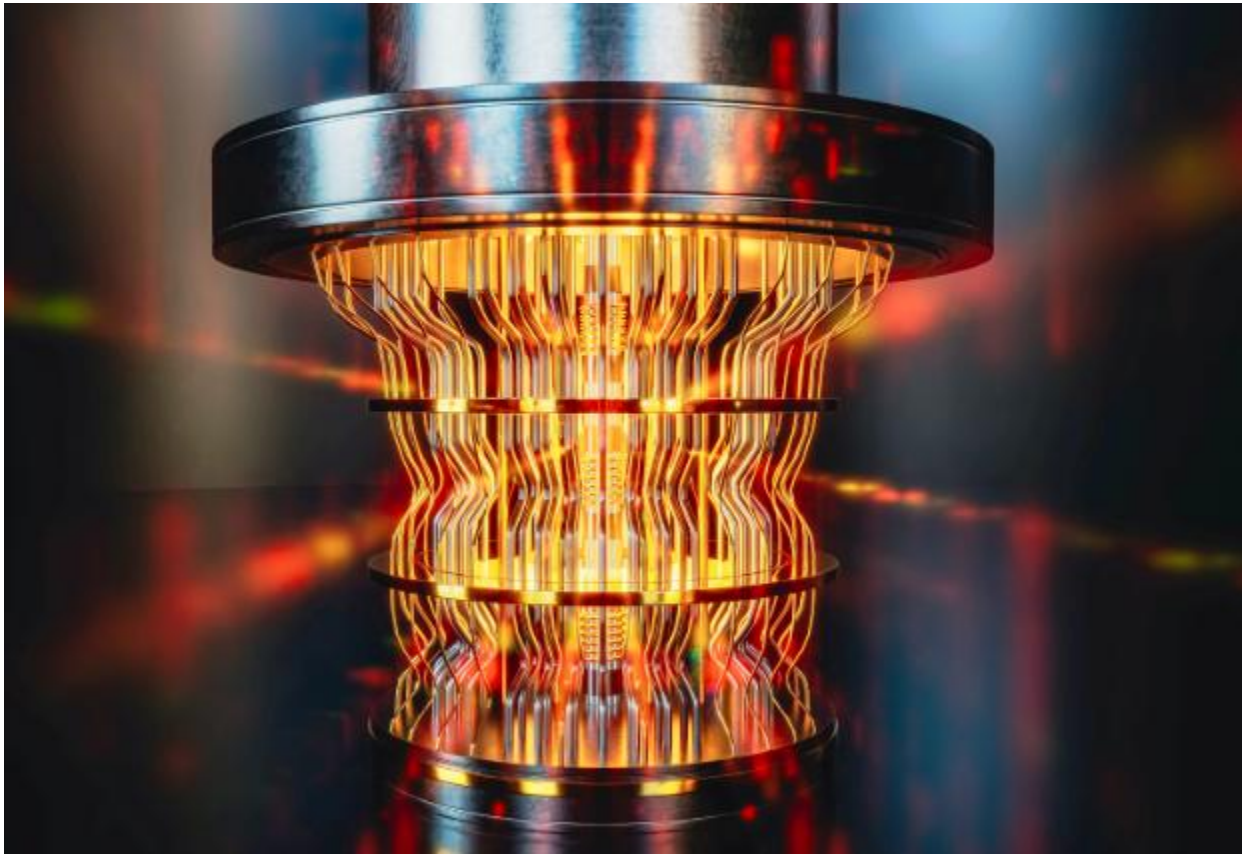




QUANTUM INFORMATION SCIENCE LAB OBSERVATION RECORD

Subject Code: VL23M04

Prepared by

Grover Quantum Centre Coordinators

Dr. V. Natarajan

Dr. G. Viieralingaam

RAJALAKSHMI INSTITUTE OF TECHNOLOGY
KUTHAMBAKKAM, CHENNAI - 600 124

BONAFIDE CERTIFICATE

NAME

ACADEMIC YEAR.....YEAR.....BRANCH.....

ROLL No.

Certified that this is the bonafide record of work done by the above student in the

.....Laboratory during the year **20** - **20**

Signature
Faculty - in - Charge

Signature
H.O.D

Submitted for the Practical Examination held on.....

Internal Examiner

External Examiner

INDEX

QUANTUM INFORMATION SCIENCE — VL23M04 (Practical Component)

S.No	Title of the Experiment	Date of Completion	Signature of Faculty
1	Study of NMR Phenomenon and Signal Analysis		
2	Create and Visualize Quantum States using a Quantum Circuit and Simulate Projective Measurements		
3	Demonstrate Quantum Circuits using Hadamard and CNOT Gates; Measure Violations of Bell Inequalities		
4	Implement and Simulate the Grover Algorithm and Decoherence Algorithm		
5	Implement and Simulate the Deutsch–Jozsa Algorithm to Distinguish Constant and Balanced Functions		
6	Demonstrate Period-Finding on Small Numbers and Outline Key Components of Shor's Algorithm		

LAB OBSERVATION RECORD

QUANTUM INFORMATION SCIENCE — VL23M04

Practical Component (Free Online Simulators, No Installation Required)

About This Manual

This manual covers all 6 practical exercises listed in the course syllabus. Each experiment can be performed using a free, browser-based simulator — no paid software, no installation, and (in most cases) no login is required. Each experiment gives you the aim, the simulator link, the background you need, a systematic procedure, a ready-to-use observation table, and points to remember.

List of Experiments

Experiment 1: Study of NMR Phenomenon and Signal Analysis

Experiment 2: Create and Visualize Quantum States using a Quantum Circuit and Simulate Projective Measurements

Experiment 3: Demonstrate Quantum Circuits using Hadamard and CNOT Gates; Measure Violations of Bell Inequalities

Experiment 4: Implement and Simulate the Grover Algorithm and Decoherence Algorithm

Experiment 5: Implement and Simulate the Deutsch–Jozsa Algorithm to Distinguish Constant and Balanced Functions

Experiment 6: Demonstrate Period-Finding on Small Numbers and Outline Key Components of Shor's Algorithm

General Instructions to Students

- Read the 'What You Should Know Before You Start' section before touching the simulator — it will save you time.
- Perform each trial carefully and note down readings immediately; do not rely on memory.
- Draw the observation table in your record note book exactly as given, and fill it up during the session.
- A stable internet connection is needed since all simulators listed here run online in the browser.
- Screenshots of your circuit/result may be pasted into your record along with the observation table, if your institution allows it.

Experiment 1: Study of NMR Phenomenon and Signal Analysis

1. Aim

To understand, using an online simulator, how nuclear spins behave in a magnetic field, how an RF pulse tips them, and how the resulting Free Induction Decay (FID) signal is formed and analysed.

2. Software / Online Simulator Required

Bloch Simulator (DRCMR) — a free, browser-based Bloch-equation simulator that needs no installation or login.

Simulator link: <https://www.drcmr.dk/BlochSimulator>

3. What You Should Know Before You Start

- A nuclear spin placed in a static magnetic field B_0 lines up and precesses around the field direction — this precession is called Larmor precession.
- An RF pulse (usually a 90-degree pulse) tips the spin away from the B_0 axis into the transverse plane.
- Once tipped, the spin keeps precessing and slowly returns to its original position — this return is called relaxation, described by two time constants T_1 and T_2 .
- The signal picked up by the receiver coil while the spin relaxes is called the Free Induction Decay (FID) — it looks like a decaying, oscillating wave.
- Studying the FID tells us the Larmor frequency, and the T_1/T_2 relaxation times of the sample.

4. Step-by-Step Procedure

1. Open the simulator link given above in any web browser (Chrome/Firefox works best; rotate a mobile screen to landscape).
2. Set a value for the static magnetic field B_0 using the on-screen slider and note the Larmor frequency shown.
3. Choose a starting condition for the spin (equilibrium, pointing along B_0).
4. Apply a 90-degree RF pulse using the pulse button and observe the spin vector tipping into the transverse plane.
5. Watch the free precession of the spin and note how the signal (FID) decays with time.
6. Change the T_1 and T_2 relaxation values from the menu and repeat the pulse to see how faster/slower relaxation changes the FID shape.
7. Record your readings in the observation table below for at least five different settings.
8. Sketch the FID waveform you observe for two of your trials in your record.

5. Observation Table

S.No	B0 Field Set	Larmor Frequency Observed	T1 / T2 Set	Nature of FID Signal Observed
1				
2				
3				
4				
5				

(Draw this table in your record note book and fill it up while you perform each run on the simulator.)

6. Result

The precession of a nuclear spin about the static field, the effect of an RF pulse, and the decay of the resulting FID signal were studied and verified using the online Bloch simulator.

7. Precautions / Points to Remember

- Change only one parameter at a time so that you can clearly link cause and effect.
- Allow the animation to fully complete before reading off values.
- Note down the units shown on screen exactly as displayed (Tesla, seconds, Hz, etc.).
- Keep the browser window large enough (or the phone in landscape mode) for the 3D view to render properly.

Experiment 2: Create and Visualize Quantum States using a Quantum Circuit and Simulate Projective Measurements

1. Aim

To build simple single-qubit and two-qubit states on a quantum circuit simulator, visualise them, and perform projective measurements to verify the expected probabilities.

2. Software / Online Simulator Required

IBM Quantum Composer — a free drag-and-drop quantum circuit builder that runs in the browser; no software installation is needed and no login is required for up to 4 qubits.

Simulator link: <https://quantum.cloud.ibm.com/composer>

3. What You Should Know Before You Start

- A qubit's state is written as $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where $|\alpha|^2$ and $|\beta|^2$ give the probability of measuring 0 or 1.
- Applying gates such as X, H (Hadamard), Y or Z changes the state of a qubit before measurement.
- A Hadamard gate on $|0\rangle$ creates an equal superposition: $(|0\rangle + |1\rangle)/\sqrt{2}$, giving 50% chance of 0 and 50% chance of 1 on measurement.
- Projective measurement collapses the qubit to either $|0\rangle$ or $|1\rangle$ and the simulator reports outcome probabilities after many repeated shots.
- The Bloch sphere and probability histogram in the Composer let you visually confirm the state you built.

4. Step-by-Step Procedure

1. Open the IBM Quantum Composer link given above (sign-in is optional for circuits up to 4 qubits).
2. Start a new circuit with one qubit, initially in state $|0\rangle$.
3. Drag an X gate onto the qubit line and run the circuit — verify the state flips to $|1\rangle$.
4. Reset and this time drag an H gate onto the qubit — observe the resulting superposition on the Bloch sphere/statevector view.
5. Add a measurement operation and run the circuit; note the probability of getting 0 and 1 from the histogram.
6. Repeat with a two-qubit circuit: apply H on qubit 0, then measure both qubits, and note the outcome probabilities.
7. Try at least four different gate combinations (X, H, Y, Z, or combinations) and record the prepared state and measured probabilities each time.

5. Observation Table

S.No	Gate(s) Applied	Expected State	P(0) Observed	P(1) Observed	Remarks
1					
2					
3					
4					

S.No	Gate(s) Applied	Expected State	P(0) Observed	P(1) Observed	Remarks
5					

(Draw this table in your record note book and fill it up while you perform each run on the simulator.)

6. Result

Different single-qubit and two-qubit quantum states were successfully prepared and visualised, and their projective-measurement probabilities were verified to match the expected theoretical values.

7. Precautions / Points to Remember

- Always reset or start a fresh circuit before building the next state, to avoid leftover gates.
- Add a measurement gate at the end of the circuit — without it, no histogram is produced.
- Run enough shots (default is fine) so that the probability histogram is reliable.
- Compare the observed probability to the theoretically expected value and note any small deviation.

Experiment 3: Demonstrate Quantum Circuits using Hadamard and CNOT Gates; Measure Violations of Bell Inequalities

1. Aim

To construct entangled Bell states using Hadamard and CNOT gates and to experimentally verify the violation of a Bell/CHSH inequality using an online simulator.

2. Software / Online Simulator Required

Quirk – Quantum Circuit Simulator — a free, browser-based drag-and-drop simulator that also has a ready-made Bell Inequality Test (CHSH) example circuit built in.

Simulator link: <https://algassert.com/quirk>

3. What You Should Know Before You Start

- Applying a Hadamard gate on qubit 0 followed by a CNOT gate (qubit 0 as control, qubit 1 as target) creates the entangled Bell state $(|00\rangle + |11\rangle)/\sqrt{2}$.
- In an entangled pair, measuring one qubit instantly fixes the outcome of the other, even though each qubit alone looks random.
- Classical (local, non-quantum) theories predict that a combination of correlations, called the CHSH value S , can never exceed 2.
- Quantum entanglement allows this value to go up to about 2.82 ($2\sqrt{2}$), which is called violation of the Bell inequality.
- Quirk's built-in 'Bell Inequality Test (CHSH)' example circuit is ready-made for this experiment — you can load it directly from the Examples menu.

4. Step-by-Step Procedure

1. Open the Quirk link given above.
2. Build a 2-qubit circuit: place an H gate on the top qubit wire, then a CNOT gate (control on top qubit, target on bottom qubit).
3. Add measurement to both qubits and observe that the only outcomes are 00 and 11, each with about 50% probability — this confirms entanglement.
4. From the Examples menu (top of page), load the ready-made 'Bell Inequality Test (CHSH)' circuit.
5. Note the measurement settings (angles) used for each qubit and record the correlation values shown by the simulator.
6. Calculate the CHSH value S from the four correlation readings using $S = E(a,b) - E(a,b') + E(a',b) + E(a',b')$.
7. Compare your calculated S with the classical limit of 2 and note whether the inequality is violated.

5. Observation Table

S.No	Circuit / Angle Setting	Correlation E Value	CHSH Value S	Classical Limit	Violated? (Y/N)
1					
2					
3					
4					

(Draw this table in your record note book and fill it up while you perform each run on the simulator.)

6. Result

The Bell state $(|00\rangle + |11\rangle)/\sqrt{2}$ was successfully created using Hadamard and CNOT gates, and the measured CHSH value was found to exceed the classical limit of 2, confirming violation of the Bell inequality.

7. Precautions / Points to Remember

- Make sure the CNOT gate's control (dot) and target (circle) are placed on the correct wires.
- Take enough repeated runs before reading the correlation/probability values.
- Note that a small numerical value near 2.82 is the quantum theoretical maximum — do not expect exactly this number from a single run.
- Keep the qubit order (top/bottom) consistent while noting down your readings.

Experiment 4: Implement and Simulate the Grover Algorithm and Decoherence Algorithm

1. Aim

To implement Grover's search algorithm on a quantum circuit simulator and observe how it amplifies the probability of the correct answer; and to study the effect of decoherence (noise) on a quantum state.

2. Software / Online Simulator Required

Quirk – Quantum Circuit Simulator — the same free browser-based simulator used earlier; it has a ready-made 'Grover Search' example and also lets you add noise/measurement to observe decoherence.

Simulator link: <https://algassert.com/quirk>

3. What You Should Know Before You Start

- Grover's algorithm searches an unsorted list of N items in about $\sqrt{N}$ steps, much faster than a classical search which needs about $N/2$ steps.
- It works by repeatedly applying an 'oracle' (which marks the correct answer) and a 'diffusion' operator (which amplifies the marked answer's probability).
- Each Grover iteration increases the probability of measuring the correct (marked) state; too many iterations can actually reduce it again, so the number of iterations matters.
- Decoherence is the loss of a qubit's clean quantum behaviour due to unwanted interaction with its environment (noise), which reduces the reliability of the answer.
- Simulating noise (bit-flip or phase-flip probability) on a circuit shows how the neat interference pattern of Grover's algorithm gets disturbed by decoherence.

4. Step-by-Step Procedure

1. Open the Quirk link and, from the Examples menu, load the ready-made 'Grover Search' circuit.
2. Identify the oracle stage (which marks the target item) and the diffusion stage in the loaded circuit.
3. Run the circuit and note the probability of the marked state after one Grover iteration.
4. Add one more repetition of the oracle + diffusion block and note how the probability of the correct state changes.
5. Continue adding iterations up to about $\sqrt{N}$ and record how probability rises, and then falls if you go beyond the optimum count.
6. Next, on a simple 1- or 2-qubit circuit, insert a noise/depolarizing display or randomly apply an extra unwanted gate to mimic decoherence.
7. Compare the clean (no-noise) output probabilities with the noisy output probabilities and note the difference.

5. Observation Table

S.No	No. of Qubits	No. of Grover Iterations	P(marked state) — No Noise	P(marked state) — With Noise
1				
2				
3				
4				
5				

(Draw this table in your record note book and fill it up while you perform each run on the simulator.)

6. Result

Grover's algorithm was implemented and shown to amplify the probability of the correct (marked) state with each iteration up to the optimum count; introducing noise was shown to reduce this probability, demonstrating the effect of decoherence.

7. Precautions / Points to Remember

- Note the optimum number of iterations (close to $\pi\sqrt{N}/4$) — going beyond this reduces accuracy, which is expected behaviour, not an error.
- Keep the oracle definition (which state is 'marked') the same throughout a set of trials for a fair comparison.
- When testing decoherence, change only the noise level between trials and keep the rest of the circuit fixed.
- Record probabilities as percentages rounded to two decimal places for easy comparison.

Experiment 5: Implement and Simulate the Deutsch–Jozsa Algorithm to Distinguish Constant and Balanced Functions

1. Aim

To build the Deutsch–Jozsa circuit on a quantum simulator and use it to determine, with a single query, whether a given hidden function is constant or balanced.

2. Software / Online Simulator Required

IBM Quantum Composer — a free browser-based quantum circuit builder, used here to construct the oracle and Deutsch–Jozsa circuit manually.

Simulator link: <https://quantum.cloud.ibm.com/composer>

3. What You Should Know Before You Start

- A function $f(x)$ is constant if it gives the same output (all 0s or all 1s) for every input, and balanced if it gives 0 for exactly half the inputs and 1 for the other half.
- Classically, distinguishing constant from balanced may need up to $2^{(n-1)}+1$ checks in the worst case; the Deutsch–Jozsa algorithm does it with just one query using quantum parallelism.
- The standard circuit: apply X then H to the ancilla (output) qubit, apply H to all input qubits, pass through the oracle for $f(x)$, then apply H again to the input qubits and measure them.
- If all input qubits measure 0, the function is constant. If any input qubit measures 1, the function is balanced.

4. Step-by-Step Procedure

1. Open the IBM Quantum Composer link and start a circuit with 2 qubits (1 input qubit q0, 1 ancilla qubit q1) for the simplest case.
2. Set the ancilla qubit q1 to $|1\rangle$ using an X gate, then apply H gates to both q0 and q1.
3. Build a 'constant' oracle first — simply do nothing (identity) so $f(x) = 0$ for all x — then apply H to q0 again and measure q0.
4. Run the circuit and note that q0 measures 0 with 100% probability, confirming the function is constant.
5. Now change only the oracle to a 'balanced' oracle by adding a CNOT from q0 to q1, keeping everything else the same.
6. Apply H to q0 again, measure, and run the circuit — note that q0 now measures 1 with 100% probability, confirming the function is balanced.
7. Repeat with a 2-input-qubit version (3 qubits total) using a different balanced oracle to reconfirm the pattern.

5. Observation Table

S.No	Oracle Used	Actual Function Type	Measurement on Input Qubit(s)	Algorithm's Conclusion	Correct? (Y/N)
1					
2					
3					
4					

(Draw this table in your record note book and fill it up while you perform each run on the simulator.)

6. Result

The Deutsch–Jozsa algorithm was implemented for both constant and balanced oracles, and in each case a single circuit run correctly identified the type of the hidden function, verifying the quantum speed-up over the classical approach.

7. Precautions / Points to Remember

- Change only the oracle part of the circuit between trials — keep the initial H gates and final measurement identical.
- Double-check that the ancilla qubit is set to $|1\rangle$ using an X gate before applying the Hadamard gates.
- For a balanced oracle, make sure every input qubit that is meant to influence $f(x)$ is actually connected to the ancilla through a CNOT.
- Record the exact bit string measured on the input qubits, not just 'balanced/constant', so your working is traceable.

Experiment 6: Demonstrate Period-Finding on Small Numbers and Outline Key Components of Shor's Algorithm

1. Aim

To use a quantum circuit simulator to find the period of a modular exponential function for small numbers, and to identify the main building blocks of Shor's factoring algorithm.

2. Software / Online Simulator Required

Quirk – Quantum Circuit Simulator — the same free browser-based simulator; it has a ready-made 'Shor Period Finding' example circuit built in.

Simulator link: <https://algassert.com/quirk>

3. What You Should Know Before You Start

- Shor's algorithm factors large numbers efficiently on a quantum computer by first turning the factoring problem into a period-finding problem.
- For a chosen number N and a random co-prime ' a ', the function $f(x) = a^x \bmod N$ repeats itself after every ' r ' steps — this repeat length ' r ' is called the period.
- The quantum part of Shor's algorithm uses superposition and the Quantum Fourier Transform (QFT) to find this period ' r ' much faster than any classical method.
- Once ' r ' is known, simple classical arithmetic (computing the GCD of $a^{(r/2)} \pm 1$ with N) gives the factors of N .
- The main building blocks are: (1) superposition of inputs, (2) modular exponentiation as a quantum oracle, (3) Quantum Fourier Transform, and (4) measurement followed by classical post-processing.

4. Step-by-Step Procedure

1. Open the Quirk link and, from the Examples menu, load the ready-made 'Shor Period Finding' circuit.
2. Identify the three stages in the loaded circuit: input superposition, modular exponentiation (oracle for $a^x \bmod N$), and the QFT block.
3. Choose a small value of N (for example $N = 15$) and a co-prime value of a (for example $a = 2, 4, 7, 8, 11, 13$) as suggested in the example.
4. Run the circuit and note the measurement outcomes on the top (input) register.
5. Convert the measured value to a fraction of 2^n (n = number of input qubits) and use continued fractions to estimate the period r .
6. Verify your value of r by checking that $a^r \bmod N = 1$ using ordinary calculation.
7. Repeat for at least three different values of ' a ' (keeping N fixed) and tabulate the periods you obtain.
8. Using the periods found, compute the GCD to find one factor of N and confirm it divides N exactly.

5. Observation Table

S.No	N Chosen	a Chosen	Measured Output	Period r Obtained	Verified ($a^r \bmod N=1$)?
1					
2					
3					
4					

(Draw this table in your record note book and fill it up while you perform each run on the simulator.)

6. Result

Period-finding was successfully demonstrated on small numbers using the quantum circuit simulator, and the periods obtained were verified classically; the key components of Shor's algorithm — superposition, modular exponentiation, QFT, and classical post-processing — were identified and studied.

7. Precautions / Points to Remember

- Choose only values of 'a' that are co-prime to N (no common factor other than 1), as the algorithm requires this.
- Use enough input qubits so that 2^n is comfortably larger than N, for an accurate period estimate.
- Remember that individual runs may sometimes give a measurement that does not directly reveal the period — repeat a few times and use continued fractions to interpret the result.
- Always verify your final period value with a direct classical calculation before recording the result.